NAME : M. Mary Sharlin

REG.NO : 192511330

COURSE CODE : CSA4310

COURSE : Internet Programming

D. Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML and HTTP interaction is observed.

Question :-

① Identify appropriate HTML 5 semantic elements missing in the structure

HTML5 semantic elements describe the purpose of webpage content

-> < header > - Defines the page heading
-> < main > - Contains the main content
-> < section > - Groups related content
-> < footer > - Contains footer information
-> < form > - Collects student registration details.

② Modify the structure to make it standards-compliant.

```
< ! DOCTYPE html >
< html lang = "en" >
< head >
< title > Symposium Registration < / title >
< / head >
< body >
< header > < h1 > Symposium < / h1 > < / header >
< main >
< form action = "/ register"
method = "POST" >
< label > Name : < / label >
< input type = "text" name = "name"
required >
```

```
<label> Email : </label>
<input type="email" name="email"
required >
<button type="submit"> submit </button>
</form>
</main>
</body>
</html>
```

③. Interpret the HTTP request generated during form submission.

When the student clicks the Register button the browser sends an HTTP POST request to the /register URL.

Example :-

```
POST /register HTTP/1.1
Host : example.com
Content-Type : application/
x-www-form-urlencoded

name = Arun & email = arun%40example.com
```

Now,

-> POST sends form data in the request body.
-> /register identifies the server endpoint.
-> Content-Type specifies the format of the submitted data.
-> The request body contains the student's name and email.

④ Analyze the HTTP response cycle from server to browser.

→ The student fills in the registration form.
→ The browser sends an HTTP POST request.
→ The server receives and validates the data.
→ The server processes the registration.
→ The browser displays a success or error message.

Common response codes include :-

→ 200 OK - Request processed successfully.
→ 201 Created - Registration successfully created.
→ 400 Bad Request - Invalid request data.
→ 500 Internal Server Error - Server-side error.

⑤ Suggest improvements for usability and accessibility

→ Use <label> elements for every input.
→ Add the required attribute to mandatory fields.
→ Use suitable input types such as email.
→ Provide clear error and success messages.
→ Ensure sufficient color contrast and readable font sizes.
→ Make the form keyboard - accessible.
→ Use HTTPS to protect data during transmission.

2). Cross - Browser Compatibility Issue :

A company website shows inconsistent layout across browser

Questions :-

① Identify improper nesting and missing closing tags

The given HTML contains the following problems

-> The <h1> element has no explicit closing tag.
-> The <p> element has no closing tag
-> The page does not declare <!DOCTYPE html>.
-> The <html> element does not specify the document language.
-> Semantic elements such as <header> and <main> are missing.
-> The page does not specify character encoding or viewport setting.

② Analyze how different browsers may interpret this structure.

Different browser may automatically correct malformed HTML in different ways.

-> Browsers may insert missing closing tags.
-> The heading and paragraph may be displayed differently than intended.
-> Default browser styles may produce inconsistent spacing.
-> Missing document-type declarations may cause quirks mode in browsers.
-> Differences in HTML parsing and CSS defaults may affect the layout.

Therefore, valid HTML and consistent CSS should be used to improve compatibility.

(13) Identify missing semantic elements (e.g. header, section).

-> <header> - Contains the main page heading.
-> <main> - Contains the primary page content.
-> <section> - Groups related information.
-> <footer> - Contains footer information.

These elements improve document structure, accessibility and maintainability

(14) Provide a corrected W3C-compliant HTML structure

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>Company</title>
</head>
<body>
<header><h1>welcome</h1></header>
<main>
<section>
<h2>About Us</h2>
<p>Welcome to our company.</p>
</section>
</main>
<footer>Copyright 2026</footer>
</body>
</html>
```

(15) Suggest techniques to ensure cross-browser compatibility

-> Use valid HTML5 and CSS3.
-> Include the HTML 5 doctype
-> Close HTML elements properly.

-> Use responsive CSS and media queries.
-> Test the website in Chrome, Firefox, Edge and Safari.
-> Use CSS resets or normalization where appropriate
-> Validate the HTML using the official W3C validator
-> Avoid browser-specific features unless properly supported.

3) Form Submission failure.

A web application fails to send form data to the server.

Question :

① Identify issues in the form configuration.

The problems are :

-> The button uses type = "button" instead of type = "submit".
-> A normal button does not submit the form automatically
-> The GET method places submitted values in the URL.
-> Sending passwords through GET is insecure.
-> The form has no labels or required-field validation.

② Analyze why the HTTP request is not triggered.

The request is not triggered because the button has type = "button".

A button with this type performs no default form-submission action. It requires JavaScript to perform an action.

Changing it to type = "submit" allows the browser to resubmit the form.

(3) Interpret the role of GET method in this scenario.

The GET method requests a resource from the server. Form values are generally appended to the URL as query parameters.

Example:-

GET /submit ? username = Arun & password = 12345
HTTP/1.1
Host : example.com

Limitations of GET :-

-> Form values appear in the URL.
-> URLs may be stored in browser history and server logs.
-> Sensitive information such as passwords should not be sent using GET.
-> GET is intended primarily for retrieving information. For login form, POST should be used instead.

(4) Propose a corrected implementation.

<!DOCTYPE html>
<html lang="en">
<head>
<title> login </title>
</head>
<body>
<form action = "/submit" method = "POST">
<label> username: </label>
<input type = "text" name = "username" required>

```
< label > Password : </label >
< input type = "password",
name = "password" required >
< button type = "submit" > Login </button >
</ form >
</ body >
</ html >
```

⑤. Suggest improvements for secure data transmission

-> Use HTTPS to encrypt data between the browser and server.
-> Use POST instead of GET for login submission
-> Validate user input on both the client and server.
-> Store passwords using a strong password-hashing algorithm.
-> Never store passwords as plaintext.
-> Use secure session management after authentication
-> Apply rate limiting to reduce repeated login attempts
-> Return appropriate error messages without exposing sensitive server details.

